

MusicPlayer

Memoria técnica del proyecto intermodular

Título

MusicPlayer: plataforma web de gestión y reproducción musical

Autor/a

Francisco Huisa y Giovanni Alejandro

Ciclo

Desarrollo de Aplicaciones Web

Centro

IES Isidra de Guzmán

Tutor

Borja Bergua

Fecha

12 de mayo de 2026

Resumen

MusicPlayer es una aplicación web orientada a la gestión de un catálogo musical y a la reproducción organizada de canciones mediante listas, favoritos e historial. El proyecto se ha construido sobre una arquitectura monolítica generada con JHipster 9.0.0, con backend Spring Boot 4.0.3, API REST, persistencia JPA, migraciones Liquibase y frontend Angular 21. El objetivo principal consiste en ofrecer una base funcional para una plataforma musical autogestionada por una organización, con usuarios autenticados, roles diferenciados y operaciones de catálogo controladas.

La solución integra autenticación mediante JSON Web Tokens, control de acceso por autoridades, entidades de dominio para artistas, álbumes, canciones, géneros, playlists, reproducciones y favoritos, además de vistas Angular para administración, edición y consulta del catálogo. La memoria describe el problema abordado, los requisitos funcionales y no funcionales, el diseño de base de datos, la arquitectura por capas, los diagramas de actividad principales, las tecnologías seleccionadas, las pruebas realizadas y las mejoras futuras. También se incluyen anexos de instalación y uso para facilitar la reproducción del entorno.

Palabras clave: MusicPlayer, Angular, Spring Boot, JHipster, JWT, Liquibase.

Abstract

MusicPlayer is a web application designed to manage a music catalogue and organise playback through playlists, likes and listening history. The project is based on a monolithic architecture generated with JHipster 9.0.0, using a Spring Boot 4.0.3 backend, REST API, JPA persistence, Liquibase migrations and an Angular 21 frontend. Its main purpose is to provide a functional foundation for a self-managed music platform operated by an organisation, with authenticated users, differentiated roles and controlled catalogue operations.

The solution integrates JSON Web Token authentication, authority-based access control, domain entities for artists, albums, songs, genres, playlists, plays and likes, as well as Angular views for administration, editing and catalogue browsing. This report describes the problem statement, functional and non-functional requirements, database design, layered architecture, main activity diagrams, selected technologies, validation tests and future improvements. Installation and user-oriented annexes are also included to make the development environment reproducible.

Keywords: MusicPlayer, Angular, Spring Boot, JHipster, JWT, Liquibase.

Índices

Índice general

1. Introducción y objetivos del proyecto
 - 1.1. Contexto y motivación
 - 1.2. Objetivos generales y específicos
 - 1.3. Alcance del trabajo
2. Análisis del problema y justificación de la solución
 - 2.1. Necesidades detectadas
 - 2.2. Requisitos funcionales
 - 2.3. Requisitos no funcionales
 - 2.4. Casos de uso principales
 - 2.5. Riesgos y decisiones de mitigación

- 2.6. Estado del arte y alternativas
- 3. Diseño del sistema
 - 3.1. Arquitectura general
 - 3.2. Modelo entidad-relación explicado
 - 3.2.1. Funcionamiento lógico de la base de datos
 - 3.2.2. Correspondencia con la rama RamaFran-Flujo/Back
 - 3.3. Diagrama de clases explicado
 - 3.3.1. Responsabilidad de las clases del dominio
 - 3.3.2. Clases técnicas de soporte
 - 3.4. Diagramas de actividad
 - 3.5. Diseño de flujos técnicos
- 4. Desarrollo
 - 4.1. Tecnologías y herramientas
 - 4.2. Backend y API REST
 - 4.2.1. Clases backend principales
 - 4.2.2. Fragmentos de código explicados
 - 4.2.3. Flujo backend de creación y publicación de canciones
 - 4.2.4. Gestión de ficheros y streaming
 - 4.3. Frontend Angular
 - 4.3.1. Clases frontend principales
 - 4.3.2. Organización de pantallas y navegación
 - 4.3.3. Estado reactivo y sincronización del reproductor
 - 4.4. Persistencia y migraciones
 - 4.4.1. Migraciones Liquibase clave
 - 4.4.2. Correspondencia entre JDL, entidades y tablas
 - 4.5. Seguridad e internacionalización
 - 4.5.1. Recorrido de una petición JWT
 - 4.6. Despliegue y ejecución
 - 4.7. Planificación temporal
 - 4.8. Estimación económica
- 5. Pruebas realizadas y validación
 - 5.1. Estrategia de pruebas
 - 5.2. Casos de prueba funcionales
 - 5.3. Validación técnica
 - 5.3.1. Evidencias automáticas disponibles
 - 5.3.2. Riesgos no cubiertos completamente
 - 5.4. Resultados por módulo
 - 5.5. Criterios de aceptación final
- 6. Conclusiones y mejoras futuras
- 7. Bibliografía y recursos consultados
- 8. Anexos

- 8.1. Anexo A: Manual de instalación
- 8.2. Anexo B: Manual de usuario
- 8.3. Anexo C: Recursos REST principales
- 8.4. Anexo D: Glosario
- 8.5. Anexo E: Matriz de trazabilidad
- 8.6. Anexo F: Checklist de calidad de la memoria
- 8.7. Anexo G: Diccionario de datos
- 8.8. Anexo H: Guía de mantenimiento
- 8.9. Anexo I: Criterios de estilo académico aplicados

Índice de figuras

- 1. Figura 1. Arquitectura general de MusicPlayer.
- 2. Figura 2. Diagrama entidad-relación lógico de la base de datos.
- 3. Figura 3. Diagrama de clases simplificado del dominio.
- 4. Figura 4. Diagrama de actividad del inicio de sesión.
- 5. Figura 5. Diagrama de actividad de creación de canción.
- 6. Figura 6. Pantalla de administración.
- 7. Figura 7. Pantalla de listado de canciones y reproductor.
- 8. Figura 8. Reproductor persistente y streaming de audio.
- 9. Figura 9. Pirámide de pruebas del proyecto.

Índice de tablas

- 1. Tabla 1. Requisitos funcionales.
- 2. Tabla 2. Requisitos no funcionales.
- 3. Tabla 3. Casos de uso principales.
- 4. Tabla 4. Entidades del dominio.
- 5. Tabla 5. Tecnologías empleadas.
- 6. Tabla 6. Estrategia de pruebas.
- 7. Tabla 7. Casos de prueba funcionales.
- 8. Tabla 8. Recursos REST principales.

9. Tabla 9. Comparativa de alternativas.
10. Tabla 10. Riesgos técnicos y mitigación.
11. Tabla 11. Planificación temporal.
12. Tabla 12. Estimación económica.
13. Tabla 13. Resultados por módulo.
14. Tabla 14. Matriz de trazabilidad.
15. Tabla 15. Checklist de calidad documental.
16. Tabla 16. Diccionario de datos.
17. Tabla 17. Operaciones de mantenimiento.
18. Tabla 18. Revisión de estilo académico.
19. Tabla 19. Responsabilidades de clases del dominio y soporte.
20. Tabla 20. Migraciones Liquibase clave.
21. Tabla 21. Correspondencia entre modelo lógico y modelo físico.
22. Tabla 22. Evidencias de validación técnica.

Índice de abreviaturas y acrónimos

Acrónimo	Significado
API	Application Programming Interface; interfaz de programación usada por el frontend para comunicarse con el backend.
CRUD	Create, Read, Update, Delete; conjunto básico de operaciones sobre entidades.
DTO	Data Transfer Object; objeto usado para transportar datos entre capas.
JWT	JSON Web Token; token firmado que identifica al usuario autenticado.
JPA	Java Persistence API; especificación de persistencia usada con Hibernate.
SPA	Single Page Application; aplicación web que carga una única página y cambia vistas desde el cliente.

Acrónimo	Significado
TFG	Trabajo de Fin de Grado o proyecto final equivalente.

1. Introducción y objetivos del proyecto

1.1. Contexto y motivación

El consumo musical actual se apoya en plataformas digitales que permiten acceder a catálogos completos desde el navegador o desde dispositivos móviles. Sin embargo, la mayoría de estas plataformas se ofrecen como servicios cerrados, gestionados por empresas externas y orientados a grandes volúmenes de usuarios. Esta situación deja un espacio para aplicaciones autogestionadas que puedan ser instaladas por una organización pequeña, un centro educativo, un colectivo cultural o un equipo de artistas que necesite publicar y organizar su propio contenido sin depender de una plataforma comercial.

MusicPlayer se plantea como una respuesta académica y técnica a ese escenario. No pretende competir con servicios de escala global, sino demostrar cómo se puede diseñar una aplicación musical completa con tecnologías actuales de desarrollo web: autenticación segura, API REST, persistencia relacional, migraciones reproducibles, interfaz responsive y separación clara entre cliente y servidor.

La motivación formativa del proyecto consiste en integrar competencias de desarrollo frontend, backend, bases de datos, seguridad y pruebas. La memoria técnica se ha redactado con un enfoque explicativo para que el lector pueda comprender por qué se ha elegido cada solución y cómo se relacionan las partes del sistema.

1.2. Objetivos generales y específicos

El objetivo general es desarrollar una plataforma web que permita gestionar un catálogo musical y ofrecer a los usuarios autenticados una experiencia básica de consulta, organización y reproducción visual del contenido. La aplicación debía organizar entidades musicales, controlar permisos por rol y mantener una base de datos reproducible mediante migraciones.

Como objetivos específicos se establecen los siguientes:

1. Diseñar un modelo de datos relacional para representar artistas, álbumes, canciones, géneros, playlists, reproducciones y favoritos.
2. Implementar una API REST que permita operar con las entidades del dominio de forma paginada, validada y segura.
3. Configurar autenticación JWT y autorización basada en roles de usuario.
4. Construir una interfaz Angular con pantallas de administración, edición y consulta para los distintos perfiles.
5. Integrar reproducción de audio y consulta de letras almacenadas dentro del propio modelo de canción.
6. Definir pruebas unitarias, de integración y funcionales que validen los flujos principales.
7. Documentar la instalación, el uso básico y los límites actuales de la solución.

1.3. Alcance del trabajo

La versión documentada incluye la generación y personalización de una aplicación JHipster con Angular y Spring Boot. Se implementan entidades de negocio, servicios, repositorios, DTO, mappers, controladores REST, rutas Angular, formularios y listados. También se incluye la gestión estándar de usuarios de JHipster, la seguridad JWT y los roles ROLE_ADMIN, ROLE_USER, ROLE_EDITOR y ROLE_ARTIST.

El alcance funcional cubre la gestión de catálogo, la organización de playlists, el registro de reproducciones, los favoritos, la visualización de letras almacenadas y una primera solución de reproducción de audio. En la rama RamaFran-Flujo/Back la subida y el streaming de archivos ya se encuentran presentes mediante FileUploadResource y la carpeta uploads, aunque su endurecimiento para producción queda identificado como mejora futura.

El proyecto se presenta como prototipo funcional y base ampliable. Por tanto, se valora especialmente la coherencia del diseño, la reproducibilidad del entorno, la claridad de la arquitectura y la trazabilidad entre requisitos, implementación y pruebas.

2. Análisis del problema y justificación de la solución

2.1. Necesidades detectadas

Una plataforma musical autogestionada necesita resolver tres grupos de necesidades. En primer lugar, necesita gestionar contenido: canciones, artistas, álbumes y géneros deben guardarse con información suficiente para poder ser buscados, ordenados y relacionados. En segundo lugar, debe permitir que los usuarios organicen ese contenido mediante playlists, favoritos e historial. En tercer lugar, debe proteger las operaciones sensibles para que la administración del catálogo no quede expuesta a cualquier usuario autenticado.

La solución elegida se justifica por la combinación de Spring Boot y Angular. Spring Boot proporciona una base robusta para exponer una API REST y aplicar seguridad. Angular facilita una interfaz modular y escalable. JHipster permite unir ambas tecnologías con una estructura inicial coherente, generando capas, pruebas y migraciones que después se adaptan al dominio musical.

2.2. Requisitos funcionales

La Tabla 1 recoge los requisitos funcionales principales. Se han redactado en términos verificables para facilitar su relación con los casos de prueba.

Table 1: Tabla 1. Requisitos funcionales.

ID	Requisito	Rol principal
RF-01	El sistema debe permitir autenticarse con usuario y contraseña y recibir un token JWT.	Todos
RF-02	El sistema debe permitir registrar usuarios y gestionar la cuenta mediante los mecanismos estándar de JHipster.	Público / usuario

ID	Requisito	Rol principal
RF-03	El sistema debe permitir crear, consultar, modificar y eliminar canciones con título, duración, URL de archivo, portada, letra y fecha de lanzamiento.	ADMIN, EDITOR, ARTIST
RF-04	El sistema debe permitir gestionar álbumes asociados a artistas y clasificados por tipo: álbum, single, EP o serie de podcast.	ADMIN, EDITOR, ARTIST
RF-05	El sistema debe permitir gestionar artistas con biografía, imagen, país y estado de verificación.	ADMIN, EDITOR
RF-06	El sistema debe permitir gestionar géneros musicales y relacionarlos con álbumes y canciones.	ADMIN, EDITOR
RF-07	El sistema debe permitir crear playlists públicas o privadas y añadir canciones con una posición determinada.	USER
RF-08	El sistema debe registrar reproducciones asociadas a usuario y canción.	USER
RF-09	El sistema debe permitir marcar canciones como favoritas mediante la entidad Like.	USER
RF-10	El sistema debe mostrar dashboards diferenciados para administración, edición y usuario final.	Todos
RF-11	El sistema debe ofrecer una barra de reproductor persistente con controles visuales de reproducción, volumen, progreso, repetición y aleatorio.	Todos
RF-12	El sistema debe permitir reproducir audio mediante el reproductor persistente y consultar letras almacenadas en la información de la canción.	Todos

2.3. Requisitos no funcionales

La Tabla 2 enumera los requisitos no funcionales. Estos requisitos condicionan las decisiones de arquitectura y validación.

Table 2: Tabla 2. Requisitos no funcionales.

ID	Requisito	Criterio de validación
RNF-01	Seguridad	Las rutas de API bajo /api/** requieren autenticación salvo autenticación, registro y recuperación de cuenta.
RNF-02	Control de acceso	Las rutas de administración requieren ROLE_ADMIN y las vistas de edición de catálogo limitan el acceso a roles autorizados.
RNF-03	Mantenibilidad	El proyecto mantiene separación entre dominio, repositorio, servicio, DTO, mapper y controlador REST.
RNF-04	Reproducibilidad	El esquema de base de datos se versiona mediante 16 changelogs Liquibase.
RNF-05	Internacionalización	La interfaz dispone de recursos en español e inglés mediante ngx-translate.
RNF-06	Usabilidad	La interfaz usa dashboards, navegación lateral, formularios validados y paginación.
RNF-07	Observabilidad	El backend incluye Actuator para salud, métricas y configuración, restringiendo endpoints sensibles.
RNF-08	Pruebas	El repositorio incluye 87 pruebas Java y 116 especificaciones frontend TypeScript.

2.4. Casos de uso principales

Los casos de uso resumen las interacciones más importantes entre actores y sistema. La Tabla 3 muestra los casos que sirven de base a los flujos explicados en el diseño.

Table 3: Tabla 3. Casos de uso principales.

Código	Actor	Descripción	Resultado esperado
CU-01	Usuario no autenticado	Iniciar sesión mediante formulario.	El sistema valida credenciales, emite JWT y redirige al dashboard correspondiente.
CU-02	Administrador	Gestionar usuarios, roles y monitorización.	Se accede a las pantallas de administración y a endpoints protegidos.
CU-03	Editor o artista	Crear o modificar contenido del catálogo.	La canción, álbum o artista queda persistido y visible en listados.
CU-04	Usuario final	Crear una playlist y añadir canciones.	La playlist queda asociada al usuario y mantiene el orden de canciones.
CU-05	Usuario final	Marcar canciones como favoritas.	Se crea o actualiza una relación Like entre usuario y canción.
CU-06	Usuario final	Reproducir una canción y consultar su información adicional.	El reproductor solicita el audio al backend y la vista muestra la letra almacenada si existe.

2.5. Riesgos y decisiones de mitigación

El principal riesgo técnico del proyecto es que el uso de un generador como JHipster produzca documentación automática poco explicada. Para mitigarlo, esta memoria separa claramente la descripción del generador de las decisiones propias del dominio. Otro riesgo es la gestión directa de ficheros multimedia en disco, mitigada mediante validación de tipos, publicación controlada del directorio uploads y comprobaciones de seguridad en el endpoint de streaming. También existe riesgo de discrepancia entre entornos de base de datos; por ello se utiliza Liquibase como mecanismo de evolución y validación del esquema.

Table 4: Tabla 10. Riesgos técnicos y mitigación.

Riesgo	Impacto	Mitigación aplicada o propuesta
Gestión local de audio e imágenes	Un fichero malicioso o una ruta manipulada podría comprometer el servidor.	Validación de tipo MIME, normalización de rutas y publicación controlada de uploads.
Desfase entre documentación y código	La memoria podría prometer funcionalidades no implementadas.	Revisión de JDL, rutas, servicios y componentes antes de redactar la versión final.

Riesgo	Impacto	Mitigación aplicada o propuesta
Exposición de operaciones sensibles	Usuarios no autorizados podrían modificar catálogo o usuarios.	JWT, reglas de Spring Security, rutas protegidas y directivas de autoridad en Angular.
Crecimiento del modelo de datos	Relaciones complejas pueden dificultar consultas y mantenimiento.	DTO, MapStruct, repositorios separados y modelo explicado con diagramas.
Migraciones inconsistentes	Un entorno podría tener un esquema distinto al esperado.	Uso de Liquibase y changelogs versionados en el repositorio.
Acoplamiento excesivo frontend-backend	Cambios en entidades podrían romper pantallas.	Comunicación por API REST y modelos TypeScript generados por entidad.

2.6. Estado del arte y alternativas

Antes de justificar MusicPlayer conviene situarlo frente a alternativas existentes. Las grandes plataformas comerciales ofrecen una experiencia muy completa, pero no están diseñadas para que una organización pequeña instale su propia instancia y controle usuarios, catálogo y permisos. Las alternativas de código abierto, por su parte, suelen centrarse en bibliotecas personales o servidores musicales ya consolidados, pero no siempre ofrecen una arquitectura didáctica que combine backend Java, frontend moderno, seguridad JWT y generación reproducible de entidades.

Table 5: Tabla 9. Comparativa de alternativas.

Solución	Ventaja principal	Limitación para el caso del proyecto	Aporte de MusicPlayer
Spotify / Apple Music	Catálogo masivo, experiencia de usuario muy pulida y recomendaciones avanzadas.	No son autogestionables y no permiten controlar una instancia propia.	Modelo instalable y adaptable a una organización concreta.
YouTube Music	Gran variedad de contenido y fuerte integración con el ecosistema Google.	Dependencia total de una plataforma externa y catálogo no controlado por la organización.	Control del catálogo y de la base de usuarios.
SoundCloud	Facilita la publicación de artistas independientes.	Sistema cerrado y menor control administrativo por parte de una entidad externa.	Roles diferenciados y administración interna.
Navidrome / Ampache	Servidores musicales de código abierto para bibliotecas propias.	Objetivos distintos y menor integración didáctica con una arquitectura Spring + Angular.	Proyecto académico con API REST, DTO, pruebas y frontend integrado.

Solución	Ventaja principal	Limitación para el caso del proyecto	Aporte de MusicPlayer
Aplicación desarrollada desde cero	Control completo del código desde el primer archivo.	Mayor tiempo de configuración de seguridad, build, migraciones y pruebas.	JHipster acelera la base técnica y permite centrarse en el dominio musical.

3. Diseño del sistema

3.1. Arquitectura general

MusicPlayer sigue una arquitectura de aplicación web monolítica con cliente Angular y servidor Spring Boot. Aunque ambos forman parte del mismo proyecto Maven/JHipster, se comunican mediante HTTP y JSON.

como una simple salida del generador. La memoria explica qué elementos se han aprovechado, cómo se han adaptado al dominio musical y qué partes requieren evolución para alcanzar una plataforma de streaming plenamente productiva.

3. Diseño del sistema

3.1. Arquitectura general

MusicPlayer sigue una arquitectura de aplicación web monolítica con cliente Angular y servidor Spring Boot. Aunque ambos forman parte del mismo proyecto Maven/JHipster, se comunican mediante HTTP y JSON. La Figura 1 muestra esta estructura general: el navegador ejecuta la SPA Angular, el backend expone la API REST, la base de datos almacena el estado persistente y el subsistema de ficheros sirve imágenes y audio desde la carpeta `uploads`.

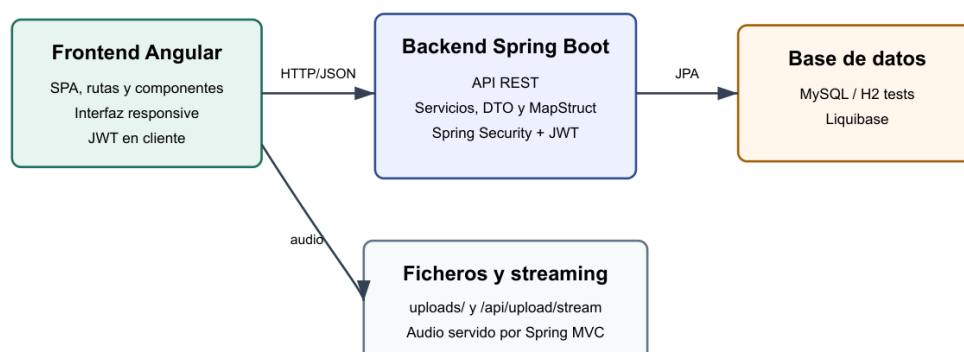


Figure 1: Figura 1. Arquitectura general de MusicPlayer. Fuente: elaboración propia.

La decisión de usar esta arquitectura se justifica por el alcance del proyecto. Un diseño de microservicios no aportaría beneficios claros para un prototipo académico y aumentaría la complejidad operativa. En

cambio, el monolito modular permite separar capas dentro de un único despliegue: controlador REST, servicio, repositorio, entidad, mapper y configuración MVC para recursos estáticos.

3.2. Modelo entidad-relación explicado

El dominio se organiza alrededor de la canción. Una canción puede pertenecer a un álbum y a un género, y puede estar asociada a varios artistas mediante una relación muchos a muchos. Las playlists se modelan con una entidad intermedia, PlaylistSong, para poder guardar la posición de cada canción dentro de la lista. Las reproducciones y favoritos relacionan usuario y canción, permitiendo construir historial y biblioteca personal.

favoritos relacionan usuario y cancion, permitiendo construir historial y biblioteca personal. En la Figura 2 se presenta primero el modelo entidad-relación lógico, más limpio y directo para explicar el diseño del sistema antes de entrar en la vista física de MySQL Workbench.

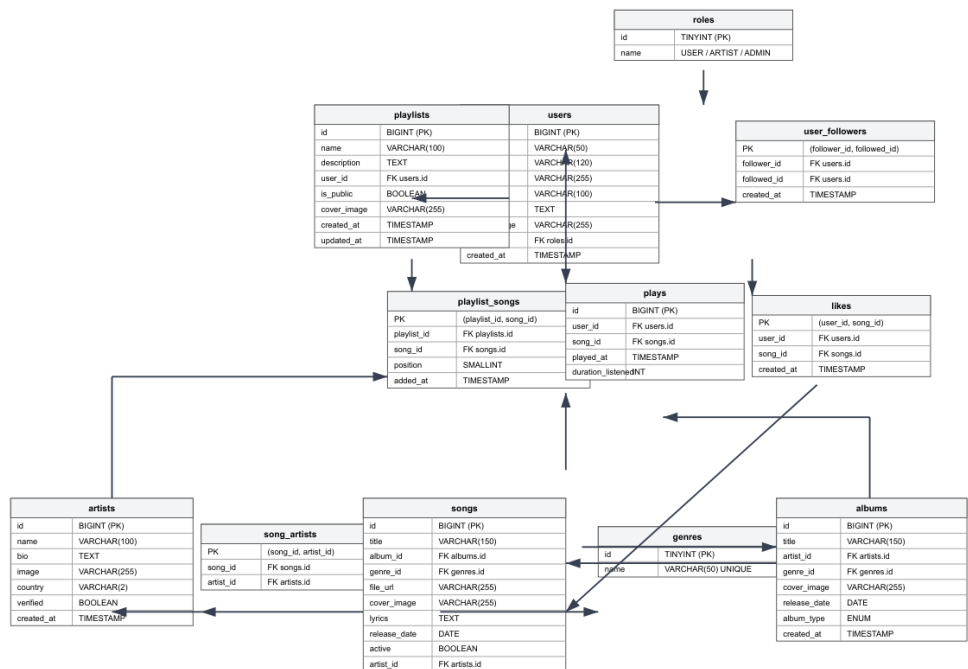


Figure 2: Figura 2. Diagrama entidad-relación lógico de la base de datos. Fuente: elaboración propia.

La Tabla 4 describe la función de cada entidad. Esta explicación evita que el diagrama quede como una imagen aislada: cada relación responde a una necesidad funcional del sistema.

Table 6: Tabla 4. Entidades del dominio.

Entidad	Responsabilidad	Relaciones principales
Genre	Clasifica canciones y álbumes mediante un nombre único.	Uno a muchos con Album y Song.
Artist	Representa artistas musicales con información descriptiva y estado de verificación.	Uno a muchos con Album y muchos a muchos con Song.

Entidad	Responsabilidad	Relaciones principales
Album	Agrupar canciones bajo un título, portada, fecha y tipo de publicación.	Muchos a uno con Artist y Genre; uno a muchos con Song.
Song	Elemento central del catálogo: título, duración, URL de archivo, portada, letra y fecha.	Muchos a uno con Album y Genre; muchos a muchos con Artist; uno a muchos con Play, Like y PlaylistSong.
Playlist	Lista creada por un usuario con visibilidad pública o privada.	Muchos a uno con User; uno a muchos con PlaylistSong.
PlaylistSong	Entidad de unión que añade posición y fecha de alta de una canción dentro de una playlist.	Muchos a uno con Playlist y Song.
Play	Registro de escucha con fecha y duración escuchada.	Muchos a uno con User y Song.
Like	Marca una canción como favorita para un usuario.	Muchos a uno con User y Song.

3.2.1. Funcionamiento lógico de la base de datos

La base de datos se puede entender en cuatro bloques. El primer bloque es identidad y permisos: usuarios y roles determinan quién accede al sistema y qué operaciones puede realizar. El segundo bloque es el catálogo musical: artistas, álbumes, géneros y canciones describen el contenido principal. El tercer bloque es la interacción del usuario con ese contenido: playlists, favoritos y reproducciones. El cuarto bloque es el soporte a relaciones complejas, representado por tablas intermedias como `playlist_songs` y `song_artists`.

El flujo normal de persistencia es el siguiente. Un usuario autenticado crea o gestiona contenido; si dispone de rol de artista o editor, la aplicación registra canciones vinculándolas a su identidad de artista. Cada canción conserva su relación con álbum, género y artista principal, mientras que las colaboraciones se representan en una tabla de unión. Cuando otro usuario escucha una canción, el sistema puede registrar una fila en `plays`; si la marca como favorita, se crea la relación correspondiente en `likes`; si la añade a una lista, se inserta una fila en `playlist_songs` con la posición concreta dentro de la playlist.

La tabla `playlist_songs` merece atención especial porque no es una unión trivial. No solo conecta playlist y canción, sino que añade atributos propios como el orden y la fecha de inserción. Esta decisión de diseño evita perder información de negocio. De forma parecida, la relación `song_artists` permite distinguir entre artista principal y artistas colaboradores sin duplicar datos en la tabla de canciones.

3.2.2. Correspondencia con la rama RamaFran-Flujo/Back

La memoria queda ligada a la rama RamaFran-Flujo/Back porque la explicación anterior se contrasta con su implementación real. En esa rama, JHipster materializa la autenticación mediante las tablas `jhi_user`, `jhi_authority` y `jhi_user_authority`, que desempeñan el mismo papel conceptual que el bloque de usuarios y roles mostrado en la figura. Del mismo modo, la tabla de relación de artistas colaboradores aparece físicamente como `rel_song_artists`, mientras que el vínculo entre artista principal y canción se refuerza con una migración específica que añade `artist_id` a `song`.

```
<addColumn tableName="song">
  <column name="artist_id" type="BIGINT" />
</addColumn>
```

```
<addForeignKeyConstraint
  baseTableName="song"
  baseColumnNames="artist_id"
  referencedTableName="artist"
  referencedColumnNames="id" />
```

El fragmento anterior, tomado de un changelog Liquibase de la rama analizada, muestra cómo evoluciona la estructura sin editar manualmente la base de datos. Este enfoque es importante en una memoria técnica porque demuestra que el modelo no solo se ha dibujado, sino que también se mantiene de forma versionada y reproducible.

3.3. Diagrama de clases explicado

```
<addforeignkeyconstraint  
  baseTableName="song"  
  baseColumnNames="artist_id"  
  referencedTableName="artist"  
  referencedColumnNames="id" />
```

El fragmento anterior, tomado de un changelog Liquibase de la rama analizada, muestra cómo evoluciona la estructura sin editar manualmente la base de datos. Este enfoque es importante en una memoria técnica porque demuestra que el modelo no solo se ha dibujado, sino que también se mantiene de forma versionada y reproducible.

3.3. Diagrama de clases explicado

La Figura 3 simplifica el diagrama de clases del dominio. No se representan todos los métodos generados por JHipster porque no aportarían comprensión al lector. En su lugar, se muestran atributos relevantes y cardinalidades. Esta decisión responde a la recomendación del tutor: el diagrama debe explicar el diseño, no convertirse en un volcado automático de clases.

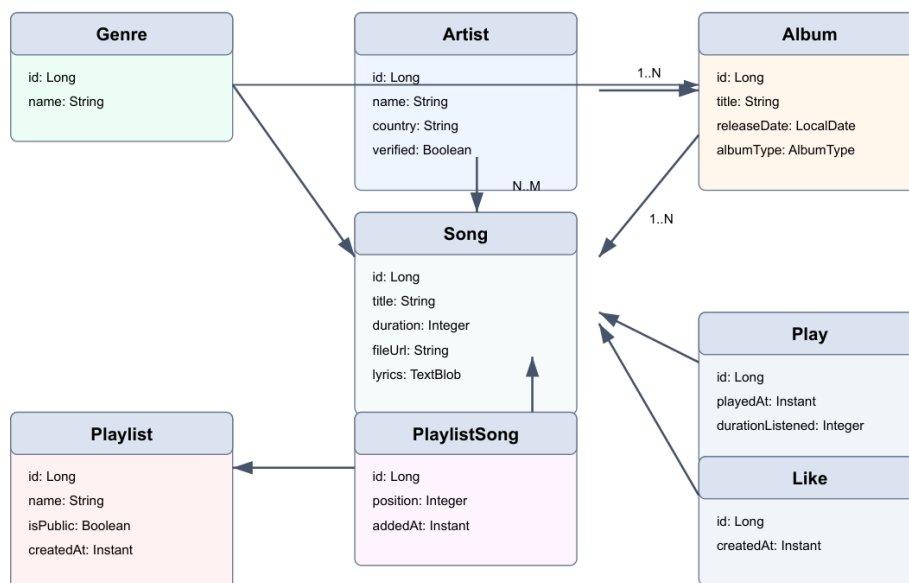


Figure 3: Figura 3. Diagrama de clases simplificado del dominio. Fuente: elaboración propia.

En la rama analizada destacan varias clases por su papel estructural. Song concentra la mayor parte del dominio musical, ya que enlaza con Album, Genre, Artist, colaboradores y datos propios como fileUrl, lyrics o active. Artist añade la relación con User, lo que permite que una canción pueda asociarse automáticamente al artista autenticado. PlaylistSong se comporta como una clase de asociación enriquecida y resuelve el problema del orden dentro de una lista.

El uso de DTO y MapStruct evita exponer directamente las entidades JPA en la API. Cada recurso REST recibe y devuelve objetos de transferencia, mientras que los mappers convierten entre DTO y entidad. Esto mejora la mantenibilidad porque permite cambiar detalles internos del modelo sin modificar necesari-

amente el contrato público. En este proyecto, esta separación también sirve para aplicar reglas de negocio en servidor, por ejemplo al impedir que el cliente asigne libremente el artista propietario de una canción.

3.3.1. Responsabilidad de las clases del dominio

La explicación del diagrama de clases no debe quedarse en una enumeración de nombres. Cada clase existe para resolver una necesidad concreta. Song actúa como agregado central del catálogo y coordina información editorial, relaciones con otras entidades y visibilidad pública. Album agrupa canciones bajo una publicación reconocible, mientras que Genre ofrece clasificación temática y facilita filtros. Artist aporta contexto autorial, biografía, imagen y vinculación con el usuario autenticado que gestiona el contenido.

Las clases Playlist, PlaylistSong, Like y Play no definen el catálogo, sino el comportamiento del usuario sobre él. Esta separación entre catálogo e interacción evita cargar la entidad canción con datos que pertenecen a la experiencia personalizada. Gracias a ello, el sistema puede escalar conceptualmente: una canción es la misma para todos, pero cada usuario mantiene su propio historial, sus favoritos y sus listas.

Table 7: Tabla 19. Responsabilidades de clases del dominio y soporte.

Clase o componente	Papel técnico	Valor dentro del proyecto
Song	Entidad principal del catálogo musical.	Concentra título, duración, fichero, letra, fecha, estado y relaciones musicales.
Artist	Entidad asociada a la autoría y gestión musical.	Permite ligar catálogo con identidad de usuario y propiedad del contenido.
Album	Agrupador editorial de canciones.	Aporta contexto de publicación y mejora la navegación del catálogo.
PlaylistSong	Clase de asociación con atributos propios.	Conserva el orden de reproducción y la fecha de inserción.
SongDTO	Contrato de transporte entre API y cliente.	Evita exponer entidades JPA completas y facilita validación.
SongMapper	Conversión entre DTO y entidad.	Centraliza el mapeo y protege campos sensibles como el artista propietario.
SongServiceImpl	Lógica de negocio de canciones.	Aplica reglas reales de publicación, propiedad y visibilidad.
SongRepository	Acceso a datos y consultas especializadas.	Resuelve filtros públicos, búsquedas y listados paginados.

3.3.2. Clases técnicas de soporte

Además del dominio puro, la aplicación depende de varias clases técnicas que unen infraestructura y negocio. DomainUserDetailsService traduce un usuario persistido a un objeto comprensible para Spring Security. AuthenticateController crea el token JWT que viaja en cada petición autenticada. FileUploadResource abstrae la entrada y salida de ficheros. Por su parte, PlayerService en Angular convierte la selección de una canción en un estado reactivo y en una reproducción real sobre la API Audio del navegador.

Estas clases no son accesorias. Son las responsables de que el modelo musical pueda utilizarse en una aplicación real. Un catálogo sin autenticación permitiría accesos indebidos; una canción sin controlador de subida no podría asociarse a un audio real; un listado sin servicio de reproducción no ofrecería la experiencia mínima esperable. Por eso la memoria incorpora su explicación y no se limita a mostrar entidades.

3.4. Diagramas de actividad

El inicio de sesión es el primer flujo crítico. Como se muestra en la figura siguiente, el usuario introduce credenciales, el backend valida la autenticación y, si es correcta, el cliente guarda el JWT y redirige según el rol. Si las credenciales son incorrectas, el formulario muestra un error sin crear sesión.

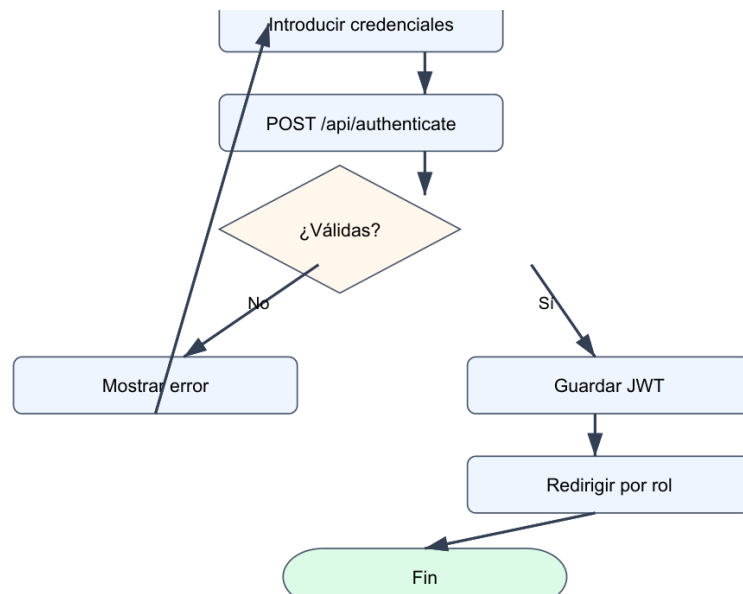


Figure 4: Figura 4. Diagrama de actividad del inicio de sesión. Fuente: elaboración propia.

El segundo flujo crítico corresponde a la creación de canciones, representado en la figura siguiente. La validación se realiza en dos niveles: el formulario Angular comprueba campos obligatorios antes de enviar la petición y el backend vuelve a validar el DTO mediante Bean Validation y seguridad por rol. Esta doble validación evita depender únicamente del cliente.

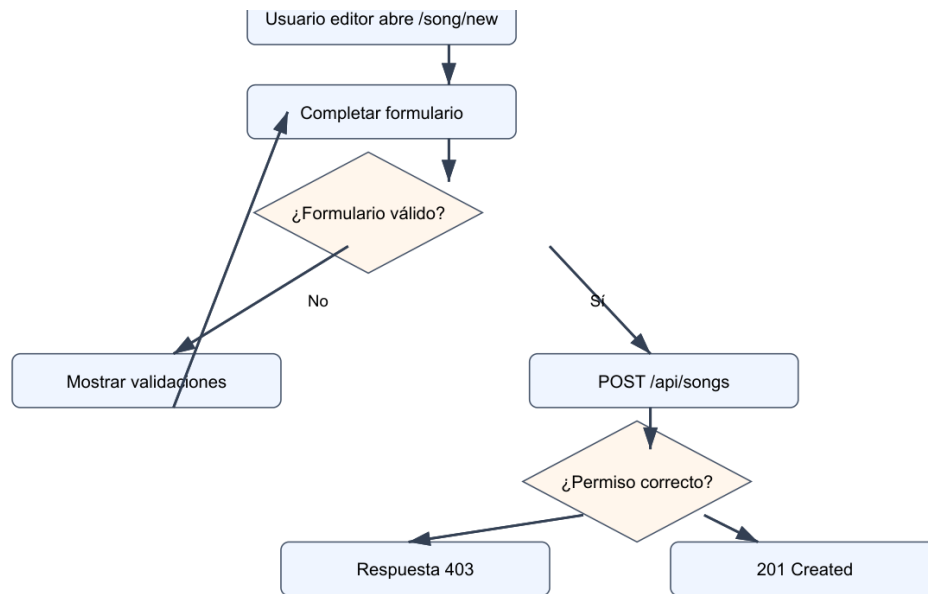


Figure 5: Figura 5. Diagrama de actividad de creación de canción. Fuente: elaboración propia.

3.5. Diseño de flujos técnicos

Más allá de los diagramas de actividad, el diseño del sistema puede entenderse como una cadena de responsabilidades. Cuando una petición nace en el navegador, pasa por un componente o servicio Angular, atraviesa interceptores, llega a un controlador REST, entra en la capa de servicio, usa repositorios y termina afectando a base de datos o sistema de ficheros. Esta secuencia explica por qué el proyecto se ha organizado en capas y por qué la memoria necesita describirlas una a una.

En el flujo de autenticación, el formulario de login recoge las credenciales, el cliente llama a `/api/authenticate`, el backend valida usuario y contraseña, emite un JWT y el cliente solicita después la identidad completa del usuario. En el flujo de creación de canción, el frontend prepara el formulario, puede subir previamente imagen y audio, envía el `SongDTO`, el servicio backend resuelve el artista propietario a partir del usuario autenticado y persiste la entidad. En el flujo de reproducción, el cliente decide la URL de audio, el backend valida la ruta y devuelve el recurso binario con el tipo MIME apropiado.

Diseñar estos recorridos de manera explícita evita varios problemas frecuentes en proyectos académicos: duplicar lógica entre frontend y backend, mezclar validación visual con validación de seguridad, o convertir el acceso a disco en una operación improvisada. La arquitectura elegida no elimina toda complejidad, pero la distribuye en puntos comprensibles y mantenibles.

4. Desarrollo

4.1. Tecnologías y herramientas utilizadas

La Tabla 5 resume las herramientas empleadas. Se han incluido únicamente las tecnologías con impacto directo sobre la arquitectura o el proceso de desarrollo.

Table 8: Tabla 5. Tecnologías empleadas.

Tecnología	Versión / uso	Justificación
JHipster	9.0.0	Generación de base Spring Boot + Angular, seguridad JWT, estructura de capas y Liquibase.
Java	21	Lenguaje del backend y versión configurada en Maven.
Spring Boot	4.0.3	Servidor REST, inyección de dependencias, seguridad, validación, Actuator y mail.
Angular	21.2.2	Frontend SPA, rutas, componentes standalone y señales reactivas.
TypeScript	5.9.3	Tipado del frontend y mejora de mantenibilidad.
MySQL	8 / configuración local	Base de datos relacional para entornos de desarrollo y producción.
Liquibase	Maven plugin y changelogs XML	Migraciones reproducibles y versionadas.
MapStruct	1.6.3	Conversión entre entidades y DTO.
Bootstrap / ng-bootstrap	5.3.8 / 20.0.0	Componentes visuales, formularios, dropdowns y paginación.
Vitest y JUnit 5	Pruebas frontend y backend	Validación automática de servicios, componentes y recursos REST.

4.2. Backend y API REST

El backend se encuentra en el paquete `com.musicplayer` y sigue la estructura habitual de JHipster. Las entidades JPA están en `domain`, los repositorios en `repository`, la lógica de aplicación en `service`, los DTO en `service.dto`, los mappers en `service.mapper` y los controladores en `web.rest`. Esta organización permite localizar rápidamente la responsabilidad de cada clase.

La API REST expone 12 recursos principales, incluyendo los recursos de negocio y los recursos de administración de usuarios/autoridades. Las operaciones de lista emplean paginación y devuelven la cabecera `X-Total-Count`, lo que permite al frontend mostrar paginadores sin descargar todos los registros. Las operaciones de creación y actualización aplican validaciones declaradas en los DTO o entidades generadas.

La configuración de seguridad declara la aplicación como `stateless`: no se almacenan sesiones de servidor, sino que cada petición autenticada incluye un token en la cabecera `Authorization`. Las rutas públicas quedan limitadas a autenticación, registro, activación, recuperación de contraseña, contenido estático y endpoints de salud. El resto de rutas bajo `/api/**` requiere autenticación.

4.2.1. Clases backend principales

`SongResource` es el punto de entrada REST para el catálogo musical. Su responsabilidad no se limita a exponer CRUD: también separa vistas públicas, vistas del artista autenticado y vistas de administración mediante rutas como `/api/songs`, `/api/songs/my-songs` y `/api/songs/admin`. Esta distinción hace visible, en la propia API, que no todos los consumidores operan sobre el mismo subconjunto de datos.

`SongServiceImpl` concentra la lógica de aplicación asociada a canciones. Aquí se decide qué artista debe quedar vinculado al registro, qué campos se pueden actualizar, cómo se filtran las canciones públicas y qué ocurre al activar o desactivar un tema. `SongRepository` encapsula consultas más específicas, por ejemplo recuperar solo canciones activas y ya publicables según su fecha de lanzamiento.

`FileUploadResource` y `StaticResourceConfig` son dos clases especialmente importantes en esta rama porque materializan una funcionalidad que afecta al uso real de la plataforma: la carga y el servido de ficheros. La primera valida y guarda audio e imágenes; la segunda publica la carpeta configurada en `app.upload.dir` bajo la ruta `/uploads/**`.

4.2.2. Fragmentos de código explicados

El primer fragmento relevante aparece en `SongMapper`:

```
@Mapping(target = "id", ignore = true)
@Mapping(target = "artist", ignore = true)
Song toEntity(SongDTO songDTO);
```

La omisión del campo `artist` no es accidental. El mapper impide que el cliente construya directamente una entidad `Song` con un artista arbitrario. De esta forma, la propiedad del contenido no queda en manos del formulario enviado por el navegador.

Esa decisión se completa en `SongServiceImpl.save`:

```
String login = SecurityUtils.getCurrentUserLogin().orElseThrow(...);
Artist artist = artistRepository.findByUserLogin(login).orElseThrow(...);
song.setArtist(artist);
song = songRepository.save(song);
```

Aquí se observa una regla de negocio real del proyecto: la canción se asigna al artista vinculado al usuario autenticado. Este comportamiento hace que la memoria explique algo más útil que un simple listado de métodos, porque muestra cómo se protege la coherencia entre identidad y catálogo.

Otro fragmento clave se encuentra en `SongRepository`:

```
@Query("SELECT s FROM Song s WHERE s.active = true
AND (s.releaseDate IS NULL OR s.releaseDate <= :today)")
```

La consulta anterior define qué canciones son públicas. No basta con que una canción exista; además debe estar activa y no tener una fecha futura de publicación. Este detalle convierte la base de datos en parte activa de la lógica de publicación y evita resolver todo en el frontend.

Por último, el backend protege el acceso al sistema de ficheros en `FileUploadResource`:

```
Path filePath = uploadPath.resolve(filename).normalize();
if (!filePath.startsWith(uploadPath)) {
    return ResponseEntity.badRequest().build();
}
```

La normalización y la comprobación de prefijo bloquean rutas manipuladas que intenten salir de la carpeta autorizada. Se trata de una comprobación sencilla, pero técnicamente significativa, porque evita exponer archivos ajenos al directorio de subida.

4.2.3. Flujo backend de creación y publicación de canciones

Cuando un usuario autorizado crea una canción, el controlador REST recibe un SongDTO. Ese DTO no se persiste directamente. Primero se transforma en entidad mediante SongMapper, después se consulta el login autenticado y, finalmente, se localiza el artista asociado a ese login. Solo entonces se rellena el campo artist y se guarda el objeto.

Esta secuencia evita una vulnerabilidad muy típica en aplicaciones CRUD: que el cliente pueda falsificar la propiedad del contenido enviando un identificador de artista distinto. El backend no confía en el formulario para decidir la autoría principal, sino que deriva esa autoría del contexto de seguridad.

La publicación pública sigue un principio parecido. Las consultas findPublicSongs y findPublicSongsByAlbumId exigen que la canción esté activa y que la fecha de lanzamiento no sea futura. Por tanto, el catálogo visible para el usuario final es un subconjunto filtrado del catálogo total. Esta decisión deja preparada la plataforma para trabajar con borradores, lanzamientos diferidos o desactivación temporal de contenidos.

El método toggleActive representa una capacidad editorial para retirar o publicar un contenido sin borrarlo físicamente. En proyectos donde el historial o las relaciones importan, esta aproximación suele ser más razonable que eliminar filas de forma permanente.

4.2.4. Gestión de ficheros y streaming

La rama analizada incorpora una pieza que ya permite hablar de reproducción real: la subida de imágenes y audios. FileUploadResource expone dos rutas de carga, una para imágenes y otra para audio. En ambos casos se valida el tipo MIME, se crea el directorio si todavía no existe y se genera un nombre aleatorio con UUID para reducir colisiones. El resultado devuelto al cliente contiene la URL pública y, en el caso del audio, también el nombre de fichero que luego utilizará el reproductor.

```
String filename = UUID.randomUUID().toString() + extension;
Path filePath = uploadPath.resolve(filename);
Files.copy(file.getInputStream(), filePath,
    StandardCopyOption.REPLACE_EXISTING);
```

En el endpoint de streaming el objetivo ya no es guardar, sino servir el recurso correcto sin abrir la puerta a rutas arbitrarias. Por eso se normaliza el nombre recibido, se comprueba que el fichero siga dentro de la carpeta autorizada y solo después se construye un UriResource. La respuesta declara además Content-Disposition: inline y Accept-Ranges: bytes, lo cual deja preparada la base para una evolución futura hacia streaming más completo.

4.3. Frontend Angular

El frontend está organizado por rutas y componentes standalone. Las pantallas de entidades se encuentran en src/main/webapp/app/entities, mientras que los servicios de autenticación, interceptores y utilidades comunes se ubican en core y shared. Esta separación replica en el cliente la modularidad del backend.

El inicio de sesión redirige a dashboards distintos según el rol del usuario. El administrador accede al panel de usuarios, catálogo, actividad y sistema; el editor accede a accesos rápidos de gestión musical; el usuario final accede a playlists, favoritos, historial y exploración del catálogo.

```
StandardCopyOption.REPLACE_EXISTING);
```

En el endpoint de streaming el objetivo ya no es guardar, sino servir el recurso correcto sin abrir la puerta a rutas arbitrarias. Por eso se normaliza el nombre recibido, se comprueba que el fichero siga dentro de la carpeta autorizada y solo después se construye un `UrlResource`. La respuesta declara además `Content-Disposition: inline` y `Accept-Ranges: bytes`, lo cual deja preparada la base para una evolución futura hacia streaming más completo.

Desde un punto de vista arquitectónico, esta solución separa bien tres responsabilidades: la base de datos conserva metadatos y relaciones, el sistema de ficheros almacena binarios y la API REST controla la exposición pública. En una memoria técnica, explicar esta triada es más valioso que copiar largas listas de propiedades, porque aclara cómo funciona de verdad la reproducción del proyecto.

4.3. Frontend Angular

El frontend está organizado por rutas y componentes standalone. Las pantallas de entidades se encuentran en `src/main/webapp/app/entities`, mientras que los servicios de autenticación, interceptores y utilidades comunes se ubican en `core` y `shared`. Esta separación replica en el cliente la modularidad del backend.

El inicio de sesión redirige a dashboards distintos según el rol del usuario. El administrador accede al panel de usuarios, catálogo, actividad y sistema; el editor accede a accesos rápidos de gestión musical; el usuario final accede a playlists, favoritos, historial y exploración del catálogo. La Figura 6 muestra una representación de la pantalla de administración basada en la interfaz implementada.

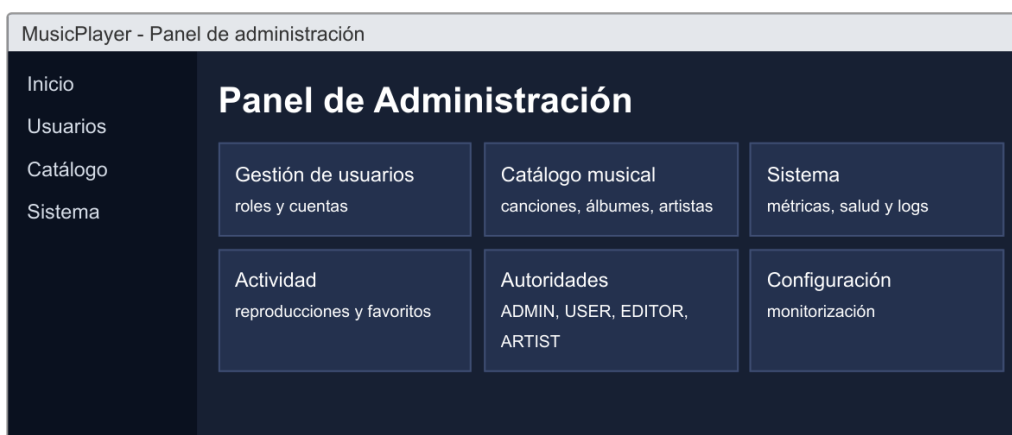


Figure 6: Figura 6. Pantalla de administración. Fuente: elaboración propia.

La lista de canciones ofrece paginación, ordenación, acciones por rol y controles visuales de reproducción. El diseño separa información musical, relaciones de álbum/género y acciones de mantenimiento.

relaciones de album/genero y acciones de mantenimiento. Los botones de creación, edición y eliminación solo se muestran a roles autorizados.

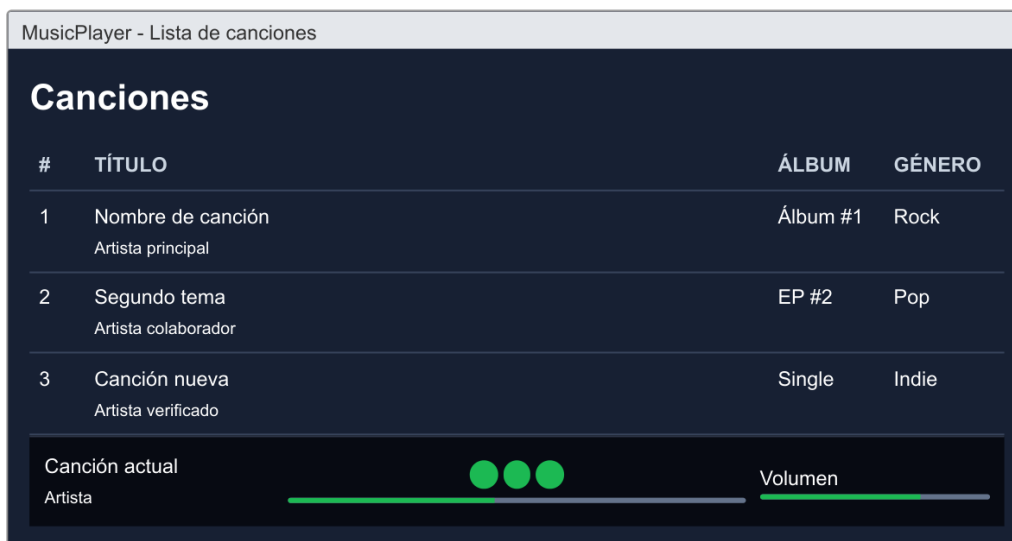


Figure 7: Figura 7. Pantalla de listado de canciones y reproductor. Fuente: elaboración propia.

4.3.1. Clases frontend principales

La clase login.ts combina formulario reactivo, señales de error y redirección por rol. Tras autenticar al usuario, consulta la cuenta actual y decide si debe abrir el dashboard de administrador, editor o usuario:

```
this.accountService.identity().subscribe(account => {
  const roles = account?.authorities ?? [];
  if (roles.includes('ROLE_ADMIN')) {
    this.router.navigate(['/dashboard-admin']);
  } else if (roles.includes('ROLE_EDITOR')) {
    this.router.navigate(['/dashboard-editor']);
  } else {
    this.router.navigate(['/dashboard-user']);
  }
});
```

El servicio login.service.ts refuerza esta idea con una implementación deliberadamente pequeña:

```
login(credentials: Login): Observable<Account | null> {
  return this.authService.login(credentials)
    .pipe(mergeMap(() => this.accountService.identity(true)));
}
```

El servicio player.service.ts mantiene estado de reproducción mediante signal, calcula iconos y progreso con computed y sincroniza el objeto Audio del navegador usando effect. Cuando una canción se selecciona, construye la URL de reproducción; si fileUrl ya es una ruta completa la reutiliza, y si no lo es, llama al endpoint de streaming del backend.

```
const fileUrl = song.fileUrl ?? '';
this.audio.src = fileUrl.startsWith('/')
  ? fileUrl
  : `/api/upload/stream/${encodeURIComponent(fileUrl)}`;
```




Figure 8: Figura 8. Reproductor persistente y streaming de audio. Fuente: elaboración propia.

4.3.2. Organización de pantallas y navegación

El frontend se organiza como una colección de rutas con responsabilidades claras. Existen pantallas de administración, edición, exploración y detalle. Esta organización mejora la mantenibilidad porque cada vista concentra un objetivo: gestionar usuarios, editar catálogo, consultar listas o reproducir contenido. A la vez, facilita el control de acceso, ya que cada ruta puede protegerse con autoridades diferentes.

4.3.3. Estado reactivo y sincronización del reproductor

El reproductor combina estado de aplicación y estado nativo del navegador. Las señales `currentSong`, `isPlaying`, `volume`, `progress` y `duration` modelan la situación lógica, mientras que el objeto `Audio` mantiene la reproducción real. El servicio actúa como puente entre ambos mundos, actualizando el estado con eventos `timeupdate`, `loadedmetadata` y `ended`.

4.4. Persistencia y migraciones

El modelo de datos se define inicialmente en `music.jdl`. A partir de esa definición se generan entidades, repositorios, servicios, DTO, mappers, controladores y changelogs. La persistencia se realiza mediante Spring Data JPA e Hibernate. Liquibase versiona el esquema para que cada entorno pueda reproducir la misma estructura sin ejecutar scripts manuales.

El repositorio contiene 16 archivos de changelog Liquibase. Esto permite evolucionar la base de datos con trazabilidad: creación de entidades, restricciones, relaciones, nuevas autoridades y ajustes posteriores como la incorporación de `artist_id` a la tabla de canciones.

4.4.1. Migraciones Liquibase clave

Table 9: Tabla 20. Migraciones Liquibase clave.

Changelog o grupo	Cambio aplicado	Justificación técnica
Creación inicial de entidades	Alta de tablas para género, artista, álbum, canción, playlist, reproducciones y favoritos.	Materializar el modelo musical definido en JDL con integridad referencial.
Relaciones muchos a muchos	Creación de tablas intermedias como <code>rel_song__artists</code> .	Representar colaboraciones musicales sin duplicar datos en canción o artista.
Autoridades personalizadas	Inserción de <code>ROLE_EDITOR</code> y <code>ROLE_ARTIST</code> .	Ampliar el esquema base de JHipster para separar consumo y edición de catálogo.
Ajuste <code>artist_id</code> en canción	Nueva columna y clave foránea desde <code>song</code> hacia <code>artist</code> .	Resolver la propiedad principal de la canción desde backend y simplificar consultas.

Changelog o grupo	Cambio aplicado	Justificación técnica
Datos y restricciones auxiliares	Índices, claves ajenas y valores por defecto.	Mantener consistencia y rendimiento mínimo en consultas frecuentes.

4.4.2. Correspondencia entre JDL, entidades y tablas

Table 10: Tabla 21. Correspondencia entre modelo lógico y modelo físico.

Nivel	Ejemplo en el proyecto	Papel en la solución
Modelo conceptual	Entidades Song, Album, Artist, Playlist y relaciones musicales.	Expresa necesidades de negocio y sirve de base para el diseño.
Definición JDL	music.jdl con entidades, campos y relaciones.	Permite generar estructura coherente de backend y frontend.
Modelo físico	Tablas song, album, artist, playlist_song, etc.	Persistencia real en MySQL con claves foráneas y restricciones.
Personalización posterior	Changelog que añade artist_id y controladores de subida/streaming.	Adapta la base generada a necesidades específicas de la rama analizada.

4.5. Seguridad e internacionalización

La seguridad combina Spring Security en el backend y guards/directivas en Angular. El backend es la fuente de verdad: aunque el cliente oculte botones, la autorización final se aplica en servidor. En el frontend, las rutas de creación y edición de canciones/álbumes requieren autoridades específicas, y la directiva `jhiHasAnyAuthority` permite ocultar acciones no permitidas.

La internacionalización se basa en archivos JSON por idioma en `src/main/webapp/i18n`. El idioma nativo es español y se incluye inglés como idioma secundario. Esta decisión facilita que las etiquetas de formularios, mensajes de error y textos de navegación puedan ampliarse sin modificar los componentes.

4.5.1. Recorrido de una petición JWT

El usuario se autentica enviando login y contraseña. El backend responde con un token firmado que contiene sujeto, autoridades e identificador de usuario. A partir de ese momento, el cliente adjunta el token en la cabecera `Authorization: Bearer ....` Spring Security intercepta la petición, valida la firma y reconstruye el contexto autenticado antes de que el controlador de negocio llegue a ejecutarse.

4.6. Despliegue y ejecución

Durante el desarrollo se ejecutan backend y frontend por separado: Spring Boot en el puerto 8080 y Angular en el puerto 4200. Para producción, Maven genera un paquete ejecutable que sirve la aplicación integrada y usa el perfil `prod`. La configuración productiva activa compresión, caché HTTP y una conexión MySQL con HikariCP.

4.7. Planificación temporal

La planificación se organizó en fases porque el proyecto combina aprendizaje tecnológico, generación inicial, personalización del dominio, desarrollo visual, pruebas y documentación. La Tabla 11 recoge una planificación temporal razonada.

Table 11: Tabla 11. Planificación temporal.

Fase	Duración estimada	Actividades principales	Resultado
Investigación y definición	2 semanas	Análisis de plataformas musicales, selección de tecnologías y definición del alcance.	Objetivos, requisitos y enfoque técnico.
Modelado del dominio	1 semana	Diseño JDL, entidades, relaciones y validaciones principales.	Modelo de datos inicial y generación de entidades.
Backend y seguridad	3 semanas	Configuración JWT, roles, recursos REST, DTO, mappers y migraciones.	API funcional y persistencia reproducible.
Frontend y experiencia de usuario	3 semanas	Dashboards, listados, formularios, navegación, reproductor visual y letras.	Interfaz navegable por roles.
Pruebas y correcciones	2 semanas	Pruebas unitarias, integración, validaciones manuales y revisión de errores.	Plan de pruebas y comportamiento validado.
Documentación final	2 semanas	Redacción de memoria, anexos, diagramas, índices, bibliografía y revisión formal.	Memoria técnica entregable en PDF/Word.

4.8. Estimación económica

La estimación económica se presenta con finalidad académica. El proyecto se ha desarrollado con herramientas gratuitas y equipos ya disponibles, por lo que no se ha producido un coste directo de licencias.

Table 12: Tabla 12. Estimación económica.

Concepto	Criterio	Coste estimado
Equipo de desarrollo	Ordenador personal ya disponible.	0 € de coste adicional.
Software base	JDK, Node.js, Spring Boot, Angular, JHipster, MySQL Community, Liquibase y VS Code/IDE comunitario.	0 € en licencias.

Concepto	Criterio	Coste estimado
Infraestructura local	Uso de entorno local y Docker para servicios auxiliares.	0 € en fase de desarrollo.
Trabajo de análisis y desarrollo	Aproximadamente 13 semanas de trabajo académico parcial.	Coste imputable al esfuerzo formativo.
Despliegue futuro	Servidor VPS básico, dominio y certificado TLS.	Pendiente de decisión; no incluido en el prototipo.

5. Pruebas realizadas y validación

5.1. Estrategia de pruebas

La validación se organiza en niveles. Las pruebas unitarias del frontend verifican servicios, formularios y componentes; las pruebas de integración del backend arrancan el contexto Spring y validan recursos REST; las pruebas funcionales manuales recorren los flujos completos desde el navegador.

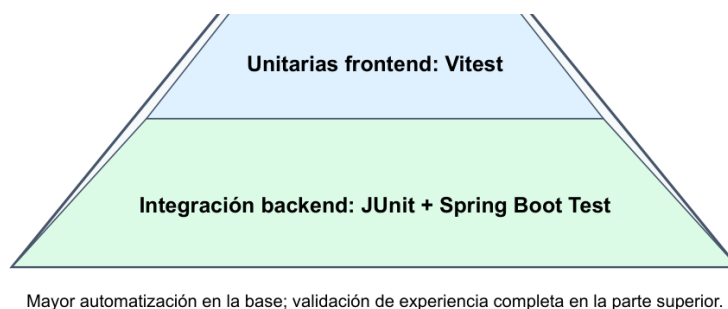


Figure 9: Figura 9. Pirámide de pruebas del proyecto. Fuente: elaboración propia.

Table 13: Tabla 6. Estrategia de pruebas.

Tipo	Herramienta	Alcance	Evidencia
Unitarias frontend	Vitest / Angular TestBed	Servicios, componentes, formularios y utilidades.	116 archivos .spec.ts.
Integración backend	JUnit 5 + Spring Boot Test	Controladores REST, repositorios, mappers y seguridad.	87 archivos Java de prueba.
Funcionales manuales	Navegador y Postman	Login, roles, CRUD, playlists, favoritos y letras.	Plan de pruebas documentado.
Regresión visual	Revisión manual en navegador	Coherencia de dashboards, listados, formularios y reproductor.	Comparación tras cambios de SCSS/HTML.

5.2. Casos de prueba funcionales

Table 14: Tabla 7. Casos de prueba funcionales.

ID	Módulo	Pasos resumidos	Resultado esperado
AUTH-01	Autenticación	Entrar en /login, introducir credenciales válidas y enviar.	Se obtiene JWT y se redirige al dashboard según rol.
AUTH-02	Autenticación	Introducir credenciales inválidas.	Se muestra error y no se crea sesión.
AUTH-03	Autorización	Acceder a una ruta de administración con usuario sin permisos.	El sistema deniega el acceso.
CAT-01	Canciones	Listar canciones autenticado.	Se muestran registros paginados y cabecera de total.
CAT-02	Canciones	Crear canción como editor con campos válidos.	La API devuelve creación correcta y la canción aparece en listado.
CAT-03	Canciones	Intentar crear canción sin título.	El formulario o backend rechaza la operación.
PL-01	Playlists	Crear playlist y asociar una canción mediante PlaylistSong.	La relación queda persistida con posición.
LIKE-01	Favoritos	Crear un Like entre usuario y canción.	La canción aparece como favorita del usuario.
LYR-01	Letras	Abrir panel de letras con canción seleccionada.	Se muestra letra o mensaje de no disponible sin romper la interfaz.
I18N-01	Internacionalización	Cambiar idioma de la interfaz.	Las etiquetas traducibles cambian entre español e inglés.

5.3. Validación técnica

La validación técnica se apoya en la compilación Maven, las pruebas generadas por JHipster, las especificaciones Angular y la comprobación manual de rutas. La existencia de pruebas no garantiza por sí sola la ausencia de defectos, pero proporciona una red de seguridad para cambios en entidades, DTO, controladores y formularios.

5.3.1. Evidencias automáticas disponibles

Table 15: Tabla 22. Evidencias de validación técnica.

Evidencia	Qué demuestra	Límite de la evidencia
Pruebas JUnit de recursos y servicios	Que controladores, mappers y repositorios mantienen contratos mínimos esperados.	No sustituyen la validación manual de permisos finos ni de experiencia de usuario.
Especificaciones Angular	Que servicios, componentes y formularios conservan comportamiento básico.	No cubren por sí solas navegación completa de extremo a extremo.
Compilación Maven/Angular	Que dependencias, tipado y configuración principal son coherentes.	Compilar no garantiza que todos los flujos de negocio estén bien resueltos.
Comprobación manual de login, catálogo y reproducción	Que los flujos principales funcionan de manera integrada.	Depende de la disciplina de ejecución y de la cobertura de escenarios elegidos.

5.3.2. Riesgos no cubiertos completamente

En esta rama siguen existiendo áreas que merecerían más validación si la aplicación evolucionara a un entorno productivo: concurrencia en subida de ficheros, límites de almacenamiento, comportamiento ante audios corruptos, soporte real de rangos HTTP, endurecimiento de permisos por entidad y automatización end-to-end de la navegación por roles.

5.4. Resultados por módulo

Table 16: Tabla 13. Resultados por módulo.

Módulo	Validación realizada	Resultado	Riesgo residual
Autenticación	Login correcto, login incorrecto, persistencia de token y redirección por rol.	Flujo principal correcto.	Revisar expiración y renovación de token en sesiones largas.
Administración	Acceso a panel, usuarios, autoridades y endpoints de gestión.	Acceso restringido a rol administrador.	Añadir auditoría visible de cambios administrativos.
Catálogo musical	CRUD de canciones, artistas, álbumes y géneros con validaciones.	Operaciones disponibles para roles autorizados.	Refinar permisos backend por entidad si se separan editor y artista.
Playlists	Creación de listas y relación ordenada con canciones.	Modelo preparado para ordenar contenido.	Implementar reglas estrictas de propiedad en todas las operaciones.
Favoritos	Relación Like entre usuario y canción.	Persistencia simple y extensible.	Evitar duplicados con restricción única usuario-canción.

Módulo	Validación realizada	Resultado	Riesgo residual
Reproducciones	Registro de Play asociado a usuario y canción.	Base para historial y estadísticas.	Añadir lógica automática al iniciar reproducción real.
Audio y reproducción	Construcción de la URL de stream, carga del fichero y actualización del estado del reproductor.	Integración funcional entre frontend y backend.	Añadir control de rangos, métricas y límites de almacenamiento.
Internacionalización	Existencia de recursos en español e inglés.	Base bilingüe disponible.	Completar traducciones de textos personalizados nuevos.

5.5. Criterios de aceptación final

Para considerar el prototipo apto como proyecto intermodular, se establecen criterios de aceptación ligados a los requisitos. Primero, la aplicación debe arrancar de forma reproducible con los comandos documentados. Segundo, un usuario debe poder autenticarse y acceder a vistas protegidas. Tercero, las entidades principales deben poder gestionarse desde API y desde interfaz. Cuarto, las relaciones entre entidades deben conservar integridad referencial. Quinto, la documentación debe permitir entender arquitectura, diseño, pruebas y despliegue sin leer directamente el código fuente.

6. Conclusiones y mejoras futuras

6.1. Conclusiones

El proyecto demuestra que es posible construir una plataforma musical autogestionada usando una pila tecnológica moderna y ampliamente utilizada. La combinación de JHipster, Spring Boot y Angular ha permitido avanzar con rapidez sin renunciar a una estructura profesional: capas separadas, seguridad JWT, migraciones Liquibase, DTO, mappers y pruebas.

La principal limitación actual es que algunas funcionalidades propias de una plataforma comercial de streaming se encuentran implementadas en una primera versión técnica, pero no alcanzan todavía un nivel productivo completo. En particular, la gestión de archivos ya dispone de carga y streaming local en backend, aunque todavía requiere endurecimiento operativo y de seguridad para un entorno real con mayor volumen.

6.2. Mejoras futuras

1. Reforzar el servicio backend de subida de audio e imágenes con límites de tamaño, antivirus, trazabilidad y borrado seguro.
2. Añadir streaming con soporte real de rangos HTTP y respuestas parciales para permitir saltos dentro del audio con mayor eficiencia.
3. Desarrollar un servicio de búsqueda por título, artista, álbum y género con filtros combinados.

4. Incorporar pruebas end-to-end con Playwright o Cypress para automatizar flujos completos de navegador.
5. Preparar un despliegue Docker completo con variables de entorno, secretos externos y configuración TLS.
6. Revisar accesibilidad: navegación por teclado, contraste, etiquetas ARIA y mensajes de validación.

7. Bibliografía y recursos consultados

Angular. (s. f.). *Angular documentation*. Recuperado el 12 de mayo de 2026, de <https://angular.dev/>

Bootstrap. (s. f.). *Bootstrap documentation 5.3*. Recuperado el 12 de mayo de 2026, de <https://getbootstrap.com/docs/5.3/>

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

JHipster. (s. f.). *JHipster documentation archive v9.0.0*. Recuperado el 12 de mayo de 2026, de <https://www.jhipster.tech/documentation-archive/v9.0.0/>

Liquibase. (s. f.). *Liquibase documentation*. Recuperado el 12 de mayo de 2026, de <https://docs.liquibase.com/>

MapStruct. (s. f.). *MapStruct reference guide*. Recuperado el 12 de mayo de 2026, de <https://mapstruct.org/documentation/stable/>

MySQL. (s. f.). *MySQL documentation*. Recuperado el 12 de mayo de 2026, de <https://dev.mysql.com/doc/>

OWASP Foundation. (2021). *OWASP Top Ten*. <https://owasp.org/www-project-top-ten/>

Pressman, R. S., & Maxim, B. R. (2019). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Spring. (s. f.). *Spring Boot reference documentation*. Recuperado el 12 de mayo de 2026, de <https://docs.spring.io/spring-boot/>

Spring Data. (s. f.). *Spring Data JPA reference documentation*. Recuperado el 12 de mayo de 2026, de <https://docs.spring.io/spring-data/jpa/reference/>

Spring Security. (s. f.). *Spring Security reference*. Recuperado el 12 de mayo de 2026, de <https://docs.spring.io/spring-security/reference/>

8. Anexos

8.1. Anexo A: Manual de instalación

8.1.1. Requisitos previos

Para ejecutar el proyecto se requiere JDK 21, Node.js compatible con el proyecto, Maven o el wrapper incluido, Git y una base de datos MySQL si se usa la configuración local actual. El repositorio incluye wrappers mvnw y npmw, por lo que no es imprescindible instalar Maven o npm globalmente si se trabaja con esos wrappers.

8.1.2. Puesta en marcha en desarrollo

```
git clone <url-del-repositorio>
cd music-player
./npmw install
docker compose -f src/main/docker/mysql.yml up --wait
```



```
./mvnw -Dskip.installnodenpm -Dskip.npm
./npmw run start
```

Con esta configuración, el backend queda disponible en `http://localhost:8080` y el frontend Angular en `http://localhost:4200`. La configuración de desarrollo del repositorio apunta a MySQL en `localhost:3307/musicplayer`. Los ficheros subidos quedan, por defecto, en la carpeta uploads definida por la propiedad `app.upload.dir`.

8.1.3. Construcción de producción

```
./mvnw -Pprod clean verify
java -jar target/*.jar
```

El perfil de producción usa MySQL en `localhost:3306/musicplayer` según la configuración actual. En un despliegue real se recomienda externalizar credenciales y secretos mediante variables de entorno o un gestor de secretos.

8.2. Anexo B: Manual de usuario

8.2.1. Inicio de sesión

El usuario accede a la pantalla de login, introduce sus credenciales y pulsa el botón de inicio de sesión. Si los datos son correctos, el sistema obtiene el perfil y lo redirige al panel que corresponde a su rol. Si son incorrectos, permanece en la pantalla de login y muestra un mensaje de error.

8.2.2. Gestión de catálogo

Los usuarios con rol autorizado pueden crear canciones, álbumes, artistas y géneros desde las rutas de entidad. Cada formulario solicita los campos obligatorios y muestra mensajes de validación cuando falta información requerida. Tras guardar, el usuario regresa al listado y puede consultar el detalle.

8.2.3. Playlists, favoritos y reproducciones

El usuario final puede navegar al catálogo, consultar canciones, crear playlists y asociar canciones a listas mediante la entidad `PlaylistSong`. También puede registrar favoritos y consultar el historial de reproducciones si dispone de datos asociados.

8.2.4. Reproducción y letras almacenadas

Desde la barra de reproducción el usuario puede iniciar, pausar, avanzar o retroceder dentro de la cola actual. El cliente solicita el audio al endpoint `/api/upload/stream/{filename}` cuando la canción se ha registrado con un nombre de fichero local. Si la canción dispone de letra almacenada en el campo `lyrics`, esta puede consultarse desde las vistas de detalle o edición sin depender de servicios externos.

8.3. Anexo C: Recursos REST principales

Table 17: Tabla 8. Recursos REST principales.

Recurso	Ruta base	Responsabilidad
<code>AuthenticateController</code>	<code>/api/authenticate</code>	Autenticación y emisión de JWT.
<code>AccountResource</code>	<code>/api/account</code>	Cuenta actual, registro, activación y contraseña.

Recurso	Ruta base	Responsabilidad
UserResource	/api/admin/users	Administración de usuarios.
AuthorityResource	/api/authorities	Gestión de autoridades del sistema.
SongResource	/api/songs	CRUD de canciones y relaciones con álbum, género y artistas.
AlbumResource	/api/albums	CRUD de álbumes y tipo de publicación.
ArtistResource	/api/artists	CRUD de artistas y verificación.
GenreResource	/api/genres	CRUD de géneros musicales.
PlaylistResource	/api/playlists	CRUD de playlists de usuario.
PlaylistSongResource	/api/playlist-songs	Asociación ordenada entre playlists y canciones.
PlayResource	/api/plays	Historial de reproducciones.
LikeResource	/api/likes	Favoritos de canciones por usuario.

8.4. Anexo D: Glosario

Autoridad: permiso asociado a un usuario en Spring Security, por ejemplo ROLE_ADMIN.
 Changelog: archivo Liquibase que describe un cambio versionado de base de datos.
 DTO: objeto de transferencia que desacopla la API de las entidades persistentes.
 Entidad: clase de dominio persistida en base de datos mediante JPA.
 Guard: mecanismo Angular que decide si una ruta puede activarse según autenticación y roles.
 Mapper: componente MapStruct que transforma entidades en DTO y viceversa.
 SPA: aplicación de una sola página que cambia de vistas en el navegador sin recargar toda la página.

8.5. Anexo E: Matriz de trazabilidad

Table 18: Tabla 14. Matriz de trazabilidad.

Requisito	Elemento de diseño	Implementación	Prueba asociada
RF-01 Autenticación JWT	Actividad de inicio de sesión y arquitectura cliente-servidor.	AuthenticateController, Spring Security y servicio de login Angular.	AUTH-01, AUTH-02, AUTH-03.
RF-03 Gestión de canciones	Entidad Song, relaciones con Album, Genre y Artist.	SongResource, SongService, rutas /songs.	CAT-01, CAT-02, CAT-03.
RF-04 Gestión de álbumes	Entidad Album y enumeración AlbumType.	AlbumResource, formulario Angular de álbum.	Pruebas CRUD de entidad y revisión manual.
RF-07 Playlists	Entidades Playlist y PlaylistSong.	PlaylistResource y PlaylistSongResource.	PL-01.
RF-09 Favoritos	Entidad Like vinculada a User y Song.	LikeResource y vistas de favoritos.	LIKE-01.

Requisito	Elemento de diseño	Implementación	Prueba asociada
RF-10 Dashboards por rol	Casos de uso de administrador, editor/artista y usuario.	Rutas /dashboard-admin, /dashboard-editor, /dashboard-user.	Validación manual de navegación por rol.
RF-12 Reproducción y letras almacenadas	Reproductor persistente, entidad Song y flujo de streaming.	player.service.ts, FileUploadResource y campo lyrics.	Validación manual de reproducción y consulta de detalle.
RNF-04 Migraciones	Modelo relacional versionado.	Changelogs Liquibase.	Arranque de backend y pruebas de integración.

8.6. Anexo F: Checklist de calidad de la memoria

Table 19: Tabla 15. Checklist de calidad documental.

Criterio solicitado	Estado en esta memoria	Observación
Portada con título, autor, tutor y fecha	Revisado	Portada sin numeración visible.
Resumen y abstract	Revisado	Resumen en español y abstract en inglés con palabras clave.
Índice general, figuras y tablas	Revisado	Secciones numeradas a dos y tres niveles.
Introducción y objetivos	Revisado	Capítulo 1.
Análisis, requisitos y casos de uso	Revisado	Capítulo 2 con tablas explicativas.
Diseño con ER, clases y actividad explicados	Revisado	Capítulo 3 con figuras referenciadas antes de aparecer.
Desarrollo, tecnologías, arquitectura y capturas	Revisado	Capítulo 4 con figuras de interfaz y explicación.
Pruebas y validación	Revisado	Capítulo 5 con estrategia, casos y resultados.
Conclusiones y mejoras futuras	Revisado	Capítulo 6.
Bibliografía APA 7	En revisión final	Referencias ordenadas alfabéticamente y con sangría francesa.
Anexos	Revisado	Instalación, usuario, REST, glosario, trazabilidad y mantenimiento.
Evitar código pegado sin sentido	Revisado	Solo se incluyen comandos mínimos de instalación.

8.7. Anexo G: Diccionario de datos

Table 20: Tabla 16. Diccionario de datos.

Entidad	Campo	Tipo	Restricción / significado
Genre	id	Long	Clave primaria generada por la base de datos.
Genre	name	String	Nombre único del género; obligatorio y limitado a 50 caracteres.
Artist	id	Long	Clave primaria del artista.
Artist	name	String	Nombre artístico; obligatorio y limitado a 100 caracteres.
Artist	bio	TextBlob	Biografía o descripción extendida.
Artist	image	String	URL o ruta de imagen representativa.
Artist	country	String	Código de país de dos caracteres.
Artist	verified	Boolean	Indica si el artista ha sido verificado por la plataforma.
Album	title	String	Título del álbum; obligatorio y limitado a 150 caracteres.
Album	coverImage	String	Imagen de portada enlazada por URL o ruta.
Album	releaseDate	LocalDate	Fecha de publicación.
Album	albumType	Enum	Clasificación: ALBUM, SINGLE, EP o PODCAST_SERIES.
Song	title	String	Título de la canción; obligatorio y limitado a 150 caracteres.
Song	duration	Integer	Duración en segundos.
Song	fileUrl	String	Referencia al archivo de audio; obligatoria en el modelo actual.
Song	coverImage	String	Imagen asociada a la canción.
Song	lyrics	TextBlob	Letra almacenada si se dispone de ella.
Song	releaseDate	LocalDate	Fecha de lanzamiento de la canción.

Entidad	Campo	Tipo	Restricción / significado
Playlist	name	String	Nombre de la lista; obligatorio y limitado a 100 caracteres.
Playlist	description	TextBlob	Descripción opcional de la playlist.
Playlist	isPublic	Boolean	Determina si la lista es visible para otros usuarios.
Playlist	coverImage	String	Imagen de portada de la playlist.
PlaylistSong	position	Integer	Orden de la canción dentro de la playlist.
PlaylistSong	addedAt	Instant	Fecha y hora de incorporación de la canción.
Play	playedAt	Instant	Momento en que se registró la reproducción.
Play	durationListened	Integer	Segundos escuchados en la reproducción.
Like	createdAt	Instant	Fecha y hora en que se marcó la canción como favorita.

8.8. Anexo H: Guía de mantenimiento

Table 21: Tabla 17. Operaciones de mantenimiento.

Operación	Procedimiento recomendado	Comprobación posterior
Añadir un campo a una entidad	Actualizar JDL o entidad, generar/crear changelog Liquibase, ajustar DTO, mapper y formulario Angular.	Compilar backend, revisar formulario y ejecutar pruebas de entidad.
Cambiar una relación	Analizar cardinalidad, impacto en datos existentes y consultas; crear migración reversible si es posible.	Validar integridad referencial y listados paginados.
Añadir un rol	Registrar autoridad en datos iniciales o changelog, actualizar constantes backend/frontend y proteger rutas.	Probar login y navegación con usuario que tenga el nuevo rol.
Modificar textos de interfaz	Actualizar archivos i18n en español e inglés.	Cambiar idioma en la aplicación y revisar que no falten claves.

Operación	Procedimiento recomendado	Comprobación posterior
Actualizar dependencias frontend	Revisar changelog de Angular/librerías, ejecutar instalación controlada y pruebas.	Ejecutar build y specs afectadas.
Actualizar Spring/JHipster	Crear rama separada, revisar breaking changes y aplicar migraciones gradualmente.	Compilar, ejecutar pruebas backend y verificar seguridad.
Preparar despliegue productivo	Externalizar secretos, configurar MySQL, revisar CORS, activar TLS y definir backups.	Prueba de arranque, login, operaciones CRUD y restauración de backup.

8.9. Anexo I: Criterios de estilo académico aplicados

Table 22: Tabla 18. Revisión de estilo académico.

Aspecto revisado	Criterio aplicado	Resultado
Persona gramatical	Uso preferente de tercera persona y pasiva refleja.	Se evita una narración excesivamente personal.
Párrafos	Extensión moderada y una idea principal por párrafo.	Mejora la legibilidad y evita bloques densos.
Figuras	Referencia en el texto antes de aparecer y pie descriptivo debajo.	Cada diagrama queda contextualizado.
Tablas	Título encima, numeración correlativa y contenido sintético.	Las tablas resumen información, no sustituyen la explicación.
Código	Uso limitado a comandos necesarios en anexos.	Se elimina el efecto de documentación automática.
Bibliografía	Formato APA 7, orden alfabético y fecha de recuperación en documentación viva.	Se cumple la pauta de citación solicitada.
Anexos	Material complementario: instalación, usuario, REST, glosario, trazabilidad y mantenimiento.	El cuerpo principal conserva coherencia y los detalles quedan separados.